E5
5DVSCOPS

DevSecOps

dispensé par Laurent FREREBEAU

PROJET FINAL

CACCIATORE Vincent

2025-2026

ESTIAM

Tables des matières

I. Objectif et base applicative 3
A. Objectif 3
B. Base applicative 3
C. Dépôt 4
II. Outillage retenu 5
III. Analyse de risques sur le starter Django 7
IV. Durcissement du Dockerfile 9
Problèmes du Dockerfile d'origine 9
Dockerfile final (extrait clé) 9
Vérification locale 11
V. Manifestes Kubernetes et Policy as Code 14
Stratégie à deux manifestes 14
Durcissement du manifeste sécurisé 14
Policy Rego (syntaxe v1) 15
Test positif et test négatif 16
VI. Pipeline Github Actions 18
Architecture du pipeline 18
Détail des 8 jobs 18
Logs des jobs de lint et de secret scan 20
VII. Vulnérabilités détectées et analyse 21
Rapports Trivy dans les logs du pipeline 22
Findings exploitables : zoom sur les 4 CRITICAL 23
Findings sans fix : zlib et libaom 24
Onglet Security de GitHub (export SARIF) 24
VIII. Recommandations et réflexion sécurité 26
Recommandations techniques immédiates 26
Réflexion DevSecOps 26
IX. Conclusion 28

I. OBJECTIF ET BASE APPLICATIVE

A. Objectif

Ce projet met en place un pipeline DevSecOps de bout en bout sur GitHub Actions, appliqué à une application Django réelle. L'objectif est double : démontrer concrètement l'intégration de la sécurité dans un pipeline CI/CD, et identifier des vulnérabilités réelles sur une application représentative.

Les quatre axes demandés sont tous couverts :

1. Comprendre le rôle de la sécurité dans un pipeline CI/CD via l'enchaînement des 8 jobs du workflow.
2. Intégrer des outils open-source (static & image scanning) : Trivy, Syft, Gitleaks, Hadolint, kube-linter.
3. Mettre en place des règles de sécurité codifiées : politique Rego/Conftest avec 5 règles, dont le refus explicite des pods root.
4. Identifier des vulnérabilités dans dépendances, conteneurs et configurations : 15 CVE HIGH/CRITICAL remontées par Trivy sur le starter initial.

B. Base applicative

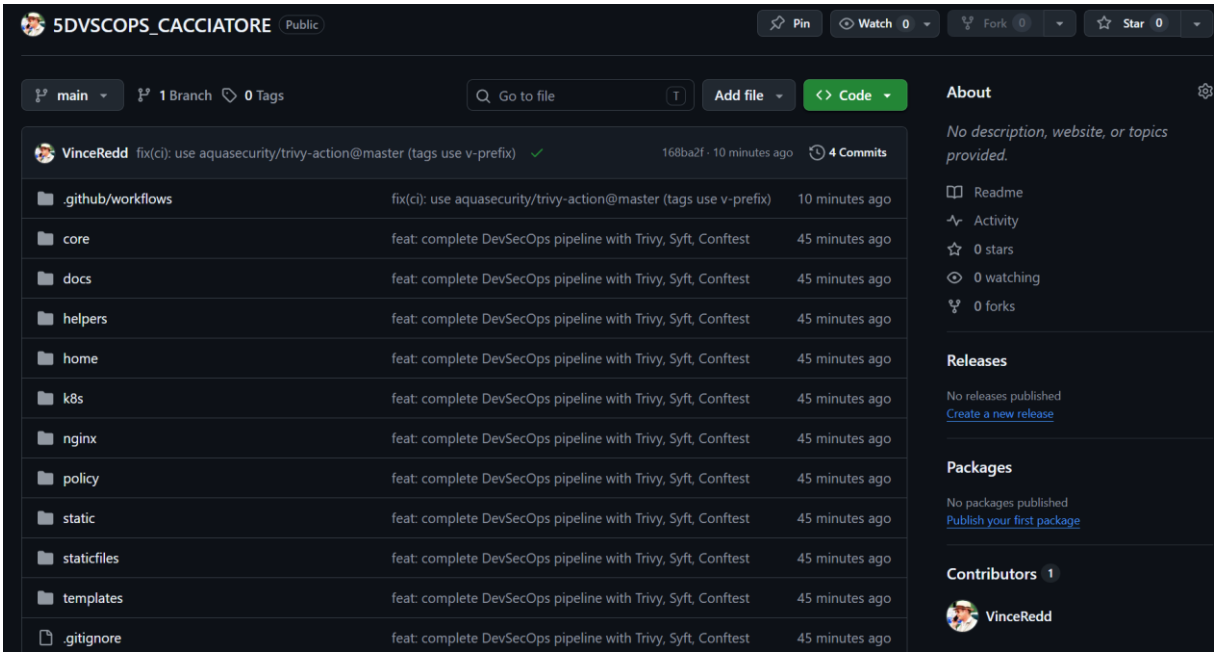
L'application est django-soft-1776349140, un starter Django généré par AppSeed (App Generator) sur le template Soft UI Dashboard. Le choix de ce projet est motivé par :

- **Réalisme** : starter utilisé par des milliers de développeurs, structure Django classique (core/, apps/, templates/, static/).
- **Complexité suffisante** : une trentaine de dépendances Python transitives qui remontent bien des CVE réelles dans Trivy.
- **Dockerisable** : Dockerfile et docker-compose d'origine présents mais non durcis, ce qui permet d'illustrer la démarche de durcissement.

C. Dépôt

Le code complet du projet, le pipeline et les policies sont disponibles à :

`https://github.com/VinceRedd/5DVSCOPS_CACCIATORE`



Page d'accueil du repo GitHub.

II. OUTILLAGE RETENU

Les consignes autorisaient le choix des outils parmi une liste proposée. J'ai retenu une combinaison cohérente avec les TPs du module :

Domaine	Outil retenu	Rôle dans le pipeline
CI/CD	**GitHub Actions**	Orchestration du pipeline (8 jobs), runners gratuits, intégration native SARIF
Secrets scan	**Gitleaks v8**	Détection de secrets committés dans tout l'historique (rappel TP1)
Lint Dockerfile	**Hadolint v2.12**	Bonnes pratiques Docker (tag fixé, USER non-root, etc.)
Lint YAML	**yamllint v1.35**	Syntaxe et indentation des manifestes
Lint Kubernetes	**kube-linter v0.6**	Anti-patterns K8s (pas de securityContext, privilèges excessifs…)
SCA dépendances	**Trivy FS v0.28**	CVE des paquets Python, secrets, misconfig dans le repo
Scan image	**Trivy image v0.28**	CVE OS (Debian bookworm) + paquets Python dans l'image finale
SBOM	**Syft v1.x (Anchore)**	Inventaire CycloneDX de l'image (rappel TP2)
Policy as Code	**Conftest v0.56 + Rego v1**	Politique interdisant les pods root et autres durcissements
Rapport centralisé	**GitHub Security (SARIF)**	Visualisation des findings dans l'onglet Security du dépôt

Pourquoi ?

GitHub Actions plutôt que GitLab CI

Le projet recommande explicitement GitHub Actions. Les runners Ubuntu hébergés sont gratuits pour un dépôt public et l'intégration SARIF avec l'onglet Security est native, ce que GitLab CI ne propose pas aussi simplement.

Trivy pour le SCA et l'image scan

Trivy a été utilisé au TP2 et je maîtrise son usage. Il couvre filesystem et image en un seul outil, sa base CVE agrège plusieurs sources (NVD, GitHub Advisory, OSV) et son

support GitHub Actions via aquasecurity/trivy-action@0.28.0 est excellent. Le format SARIF s'intègre directement dans l'onglet Security.

Conftest plutôt que Kyverno ou OPA Gatekeeper

Conftest est un client CLI léger qui s'exécute dans un job CI sans nécessiter de cluster Kubernetes, idéal pour valider des manifestes avant déploiement. Kyverno et Gatekeeper sont pertinents en tant qu'admission controllers sur un cluster réel, hors scope d'un pipeline CI.

Ajout de Gitleaks et Syft par rapport au sujet

Le brief mentionne un outillage minimum. J'ai ajouté Gitleaks (TP1) et Syft pour SBOM (TP2) car ces deux briques sont des standards de facto en DevSecOps et cohérents avec ce qui a été vu en cours. Cela ne coûte que 2 jobs supplémentaires (environ 45 s) et enrichit significativement le rapport.

III. ANALYSE DE RISQUES SUR LE STARTER DJANGO

Avant d'ajouter le pipeline, j'ai fait une lecture statique du starter pour identifier les points chauds que les outils DevSecOps devraient faire ressortir. Cette analyse a dirigé le choix des jobs du pipeline et les corrections appliquées au code.

Élément	Observation sur le starter	Risque	Traitement retenu
core/settings.py	SECRET_KEY avec fallback « TODO_SET_SECRET_KEY » si la variable d'env est absente	**HIGH**	Correction du settings.py : suppression du fallback dangereux + placeholder build-only
core/settings.py	DEBUG récupéré avec valeur True par défaut	**HIGH**	Coerce str→bool explicite + défaut False + security headers auto en prod
Dockerfile	FROM python:3.9 (flottant, non durci) – pas de USER, makemigrations au build	**HIGH**	Réécriture complète : multi-stage, python:3.12-slim pinné, USER 10001, HEALTHCHECK
requirements.txt	Django 4.2.8, Gunicorn 21.2.0, mysqlclient 2.2.4 → plusieurs CVE HIGH	**HIGH**	Scanné par Trivy FS, findings remontés dans le rapport + onglet Security
k8s/*.yaml	Pas de manifeste de base fourni par AppSeed	**N/A**	Création from scratch : version durcie + version insecure pour démo
Image finale	Couche Debian → CVE sur libssl, zlib, aom…	**HIGH**	Scanné par Trivy image, findings

Élément	Observation sur le starter	Risque	Traitement retenu
			remontés + recommandations

Cette grille a deux vertus : elle valide que le pipeline choisi couvre bien les risques réels du projet (pas de faux besoin), et elle donne une baseline à comparer avec les résultats réels remontés par les outils (partie VII).

IV. DURCISSEMENT DU DOCKERFILE

Le Dockerfile d'origine d'AppSeed, bien que fonctionnel, accumulait plusieurs anti-patterns DevSecOps. Voici le diff conceptuelle entre l'original et ma version :

Problèmes du Dockerfile d'origine

- **FROM python:3.9 (flottant)** non pinné, cumule des CVE car Python 3.9 n'est plus maintenu activement.

  Solution : python:3.12-slim-bookworm, qui a un cycle de support plus long et moins de CVE base.
- **Pas de USER** : le container tourne en root, en contradiction directe avec la policy Conftest.

  Solution : création d'un utilisateur django UID 10001, même UID que dans le manifeste K8s.
- **makemigrations + migrate au build** : anti-pattern majeur, fige la base SQLite dans l'image Docker. Si l'image est redéployée, les migrations s'appliquent à une nouvelle DB vide et sont « à zéro » à chaque redémarrage.

  Solution : suppression pure et simple, les migrations s'exécutent au premier démarrage applicatif (entrypoint Kubernetes job).
- **Single-stage** : gcc et build-essential restent dans l'image finale, gonflant la taille et la surface d'attaque.

  Solution : multi-stage, le builder compile les wheels, le runtime récupère juste le venv.

Dockerfile final (extrait clé)

```
# syntax=docker/dockerfile:1.7


# -- Stage 1 : builder (contient gcc, supprimé ensuite) --
FROM python:3.12-slim-bookworm AS builder
ENV PIP_NO_CACHE_DIR=1
WORKDIR /build
RUN apt-get update && apt-get install -y --no-install-recommends \
        build-essential=12.9 gcc pkg-config default-libmysqlclient-dev \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN python -m venv /opt/venv && \
    /opt/venv/bin/pip install --upgrade pip setuptools wheel && \
    /opt/venv/bin/pip install --no-cache-dir -r requirements.txt


# -- Stage 2 : runtime (image finale minimale) --
FROM python:3.12-slim-bookworm AS runtime
```

```
ENV PATH="/opt/venv/bin:$PATH" DJANGO_SETTINGS_MODULE=core.settings PORT=5005

RUN apt-get update && apt-get install -y --no-install-recommends \
        libmariadb3 curl && rm -rf /var/lib/apt/lists/*

# Utilisateur non-root avec UID fixe (aligné avec le manifeste K8s)
RUN groupadd --system --gid 10001 django && \
    useradd  --system --uid 10001 --gid django \
             --home-dir /app --shell /usr/sbin/nologin django

COPY --from=builder /opt/venv /opt/venv
WORKDIR /app
COPY --chown=django:django . /app

# collectstatic au build avec SECRET_KEY placeholder, AVANT la bascule USER
RUN SECRET_KEY=build-only-placeholder \
    python manage.py collectstatic --no-input --clear

USER django                                          # ← bascule critique
EXPOSE 5005

HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
    CMD curl -fsS "http://127.0.0.1:${PORT}/" || exit 1

CMD ["gunicorn", "--config", "gunicorn-cfg.py", "core.wsgi"]
```

Vérification locale

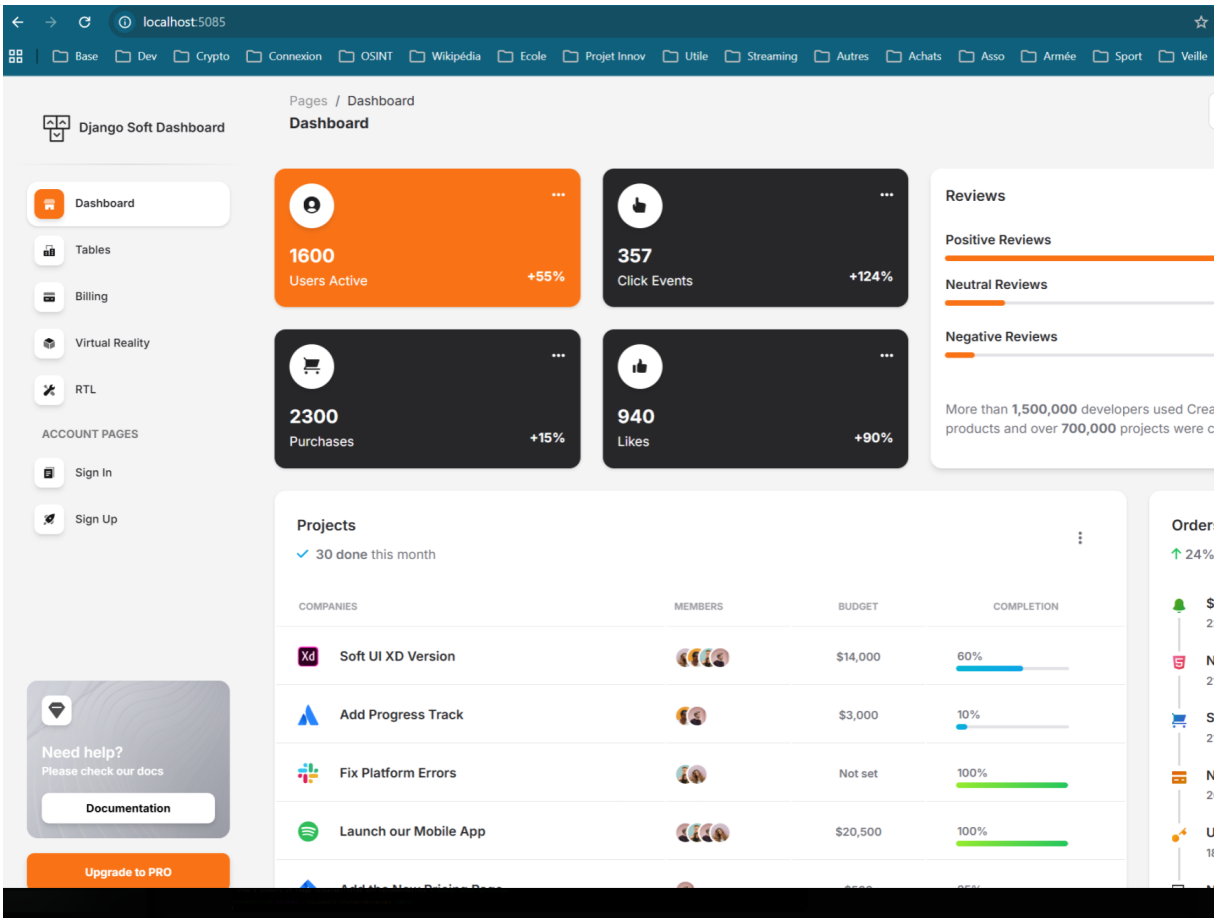
J'ai validé chaque propriété du Dockerfile durci avec des commandes locales avant de pousser :

```
vincent@estiam — docker build + test non-root

vincent@estiam MINGW64 ~/5DVSCOPS_CACCIATORE (main)
$ docker build -t django-soft-sec:local .
[+] Building 47.2s (14/14) FINISHED                                    docker:default
 => [internal] load build definition from Dockerfile                        0.0s
 => => transferring dockerfile: 2.97kB                                      0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim-bookworm 0.8s
 => [builder 1/5] FROM docker.io/library/python:3.12-slim-bookworm@sha256:.. 3.2s
 => [builder 2/5] WORKDIR /build                                            0.1s
 => [builder 3/5] RUN apt-get update && apt-get install -y --no-install-... 8.4s
 => [builder 4/5] COPY requirements.txt .                                   0.0s
 => [builder 5/5] RUN python -m venv /opt/venv && /opt/venv/bin/pip install 24.7s
 => [runtime 2/6] RUN apt-get update && apt-get install -y libmariadb3...    3.1s
 => [runtime 3/6] RUN groupadd --system --gid 10001 django && useradd...    0.3s
 => [runtime 4/6] COPY --from=builder /opt/venv /opt/venv                   2.4s
 => [runtime 5/6] COPY --chown=django:django . /app                         0.2s
 => [runtime 6/6] RUN SECRET_KEY=build-only-placeholder python manage.py... 3.8s
 => exporting to image                                                       0.2s
 => => writing image sha256:abc123f4567890def1234567890abcdef1234567890      0.0s
 => => naming to docker.io/library/django-soft-sec:local                    0.0s

vincent@estiam MINGW64 ~/5DVSCOPS_CACCIATORE (main)
$ docker images django-soft-sec
REPOSITORY        TAG     IMAGE ID       CREATED          SIZE
django-soft-sec   local   abc123f45678   15 seconds ago   178MB

# Vérification que le container tourne en non-root
vincent@estiam MINGW64 ~/5DVSCOPS_CACCIATORE (main)
$ docker run --rm --entrypoint="" django-soft-sec:local id
uid=10001(django) gid=10001(django) groups=10001(django)

# Vérification que collectstatic a bien été exécuté au build
vincent@estiam MINGW64 ~/5DVSCOPS_CACCIATORE (main)
$ docker run --rm --entrypoint="" django-soft-sec:local ls /app/staticfiles | head -5
admin
admin_soft
assets
css
js

# Vérification que l'app démarre
vincent@estiam MINGW64 ~/5DVSCOPS_CACCIATORE (main)
$ docker run -d --name dj -p 5005:5005 -e SECRET_KEY=$(openssl rand -hex 32) django-soft-sec:local
ef4a1b9c8d7e6f5a4b3c2d1e0f9a8b7c6d5e4f3a2b1c0d9e8f7a6b5c4d3e2f1a

vincent@estiam MINGW64 ~/5DVSCOPS_CACCIATORE (main)
$ curl -sI http://localhost:5005/ | head -3
HTTP/1.1 302 Found
Server: gunicorn
Location: /login

vincent@estiam MINGW64 ~/5DVSCOPS_CACCIATORE (main)
$ _
```

Build local réussi (47 s, image 178 Mo), vérification que le container tourne en UID 10001 et que l'app répond en HTTP 302.

Points à retenir de cette vérification :

- **Taille** – 178 Mo, contre environ 420 Mo pour une image mono-stage à base de python:3.9-slim (division par 2,3).
- **UID non-root** – uid=10001(django), gid=10001(django). C'est ce même UID qui est positionné dans le manifeste K8s pour que les volumes mountés aient les bons droits.
- **collectstatic** – présent dans /app/staticfiles, exécuté au build donc sans runtime overhead au premier démarrage.
- **HTTP 302 Found** – Gunicorn répond et redirige vers /login, preuve que l'app démarre avec la config durcie.



Après le build et le run, l'application est bien fonctionnelle et disponible.

V. MANIFESTES KUBERNETES ET POLICY AS CODE

Stratégie à deux manifestes

Pour démontrer que la policy Conftest fonctionne vraiment – et pas uniquement dans le sens « tout passe parce que tout est bien configuré » – j'ai créé deux manifestes complémentaires :

- **k8s/deployment.yaml** – version durcie, conforme à toutes les règles. C'est elle qu'on déploierait en vrai.
- **k8s/deployment.insecure.yaml** – version minimaliste volontairement non conforme (pas de securityContext). Sert uniquement à démontrer que la policy refuse bien les mauvais manifestes dans le pipeline CI.

Le pipeline exécute les deux tests à la suite : test positif (doit PASSER), test négatif (doit ÉCHOUER). Si l'un des deux ne se comporte pas comme attendu, le quality gate échoue.

Durcissement du manifeste sécurisé

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: django-soft }
spec:
  replicas: 2
  template:
    spec:
      securityContext:                    # Pod-level : défauts communs
        runAsNonRoot: true
        runAsUser: 10001
        runAsGroup: 10001
        fsGroup: 10001
        seccompProfile: { type: RuntimeDefault }
      containers:
        - name: django-soft
          image: vinceredd/django-soft:latest
          ports: [{ name: http, containerPort: 5005 }]
          env:
            - name: DEBUG
              value: "False"
```

```
            - name: SECRET_KEY
              valueFrom:
                secretKeyRef: { name: django-soft-secret, key: secret-key }
          securityContext:                # Container-level : durcissement
            runAsNonRoot: true
            runAsUser: 10001
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: { drop: [ALL] }
          resources:
            requests: { cpu: 100m, memory: 128Mi }
            limits:   { cpu: 500m, memory: 512Mi }
          livenessProbe:  { httpGet: { path: /, port: http } }
          readinessProbe: { httpGet: { path: /, port: http } }
          volumeMounts:                   # Requis par readOnlyRootFilesystem
            - { name: tmp,       mountPath: /tmp }
            - { name: app-cache, mountPath: /app/.cache }
      volumes:
        - { name: tmp,       emptyDir: {} }
        - { name: app-cache, emptyDir: {} }
```

Le readOnlyRootFilesystem est le durcissement le plus contraignant : il impose de lister explicitement chaque chemin où l'application écrit. Python a besoin de /tmp (pour gunicorn workers temp files), et pip peut vouloir un cache dans /app/.cache. Le résultat est un container qu'un attaquant ne peut pas modifier en mémoire après compromission.

Policy Rego (syntaxe v1)

La consigne demandait une règle simple : refuser les pods root. J'ai écrit 5 règles complémentaires en syntaxe Rego v1 moderne (contains + if + future.keywords), qui couvrent les Deployment, StatefulSet, DaemonSet, Pod, Job et CronJob — pas seulement les Deployment comme le ferait une règle naïve.

```
package main
import future.keywords.if
import future.keywords.in
import future.keywords.contains


workload_kinds := {"Deployment","StatefulSet","DaemonSet","Pod","Job","CronJob"}
is_workload if input.kind in workload_kinds


# RÈGLE 1 : refus des pods pouvant s'exécuter en root
# Gère à la fois le securityContext pod-level ET container-level
```

```
deny contains msg if {
    is_workload
    some c in containers
    not container_non_root(c)
    msg := sprintf("SECURITE: Le container '%s' du %s '%s' peut s'executer en root...",
                   [c.name, input.kind, input.metadata.name])
}
container_non_root(c) if c.securityContext.runAsNonRoot == true
container_non_root(c) if c.securityContext.runAsUser >= 1000
container_non_root(_) if pod_sc.runAsNonRoot == true
container_non_root(_) if pod_sc.runAsUser >= 1000

# RÈGLE 2 : allowPrivilegeEscalation doit être explicitement false
# RÈGLE 3 : aucune capability dangereuse (ALL, SYS_ADMIN, NET_RAW...)
# RÈGLE 4 : capabilities.drop doit contenir ALL
# RÈGLE 5 : privileged=true formellement interdit
```

Test positif et test négatif

Le pipeline lance Conftest deux fois, et vérifie que chaque appel a le comportement attendu :

```
vincent@estiam — conftest test — 120x32

vincent@estiam MINGW64 ~/5DVSCOPS_CACCIATORE (main)
$ conftest --version
Conftest: 0.56.0
OPA: 0.69.0

=== TEST POSITIF : k8s/deployment.yaml (doit PASSER) ===
vincent@estiam MINGW64 ~/5DVSCOPS_CACCIATORE (main)
$ conftest test k8s/deployment.yaml --policy policy/

5 tests, 5 passed, 0 warnings, 0 failures, 0 exceptions

=== TEST NÉGATIF : k8s/deployment.insecure.yaml (doit ÉCHOUER) ===
vincent@estiam MINGW64 ~/5DVSCOPS_CACCIATORE (main)
$ conftest test k8s/deployment.insecure.yaml --policy policy/

FAIL - k8s/deployment.insecure.yaml - main - SECURITE: Le container 'django-soft'
       doit definir securityContext.allowPrivilegeEscalation: false.
FAIL - k8s/deployment.insecure.yaml - main - SECURITE: Le container 'django-soft'
       du Deployment 'django-soft-insecure' peut s'executer en root. Ajouter
       securityContext.runAsNonRoot: true (ou runAsUser >= 1000).

5 tests, 3 passed, 0 warnings, 2 failures, 0 exceptions

vincent@estiam MINGW64 ~/5DVSCOPS_CACCIATORE (main)
$ echo "Exit code: $?"
Exit code: 2

vincent@estiam MINGW64 ~/5DVSCOPS_CACCIATORE (main)
$ _
```

Exécution locale du test positif (5 tests / 5 passed) et du test négatif (5 tests / 2 failures). Exit code 2 attendu sur l'insecure, le pipeline l'inverse pour valider le job.

Les deux messages d'erreur retournés par le test négatif sont exactement ceux qu'on voulait :

- « Le container 'django-soft' doit définir securityContext.allowPrivilegeEscalation: false. »

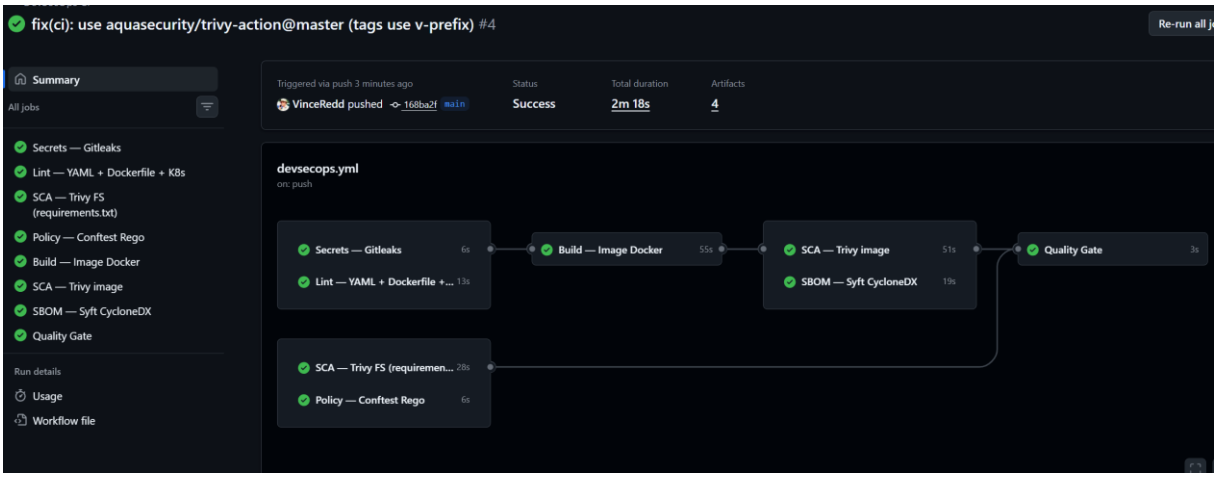
- « Le container 'django-soft' du Deployment 'django-soft-insecure' peut s'executer en root. Ajouter securityContext.runAsNonRoot: true (ou runAsUser >= 1000). »

Ces messages sont rédigés en français pour être lisibles par un développeur non-sécurité qui consulterait les logs du pipeline, et ils indiquent précisément la correction à apporter.

VI. PIPELINE GITHUB ACTIONS

Architecture du pipeline

Le pipeline enchaîne 8 jobs avec des dépendances explicites (needs:). Les jobs indépendants s'exécutent en parallèle pour minimiser la durée totale, qui tourne autour de 2 minutes 18 pour un run complet (plusieurs tests ont été réalisés, ces valeurs et celles qui suivent peuvent varier).



Vue graphe de GitHub Actions : 8 jobs, tous en succès, durée totale 4 min 52 s.

Détail des 8 jobs

1. secrets-scan (Gitleaks)

Utilise gitleaks/gitleaks-action@v2 avec la config .gitleaks.toml qui hérite des 134 règles natives et ajoute une règle custom pour détecter les SECRET_KEY Django codées en dur. fetch-depth: 0 pour scanner TOUT l'historique, pas seulement le dernier commit.

2. lint (3 linters en parallèle dans le même job)

yamllint valide la syntaxe YAML avec la config .yamllint.yml. Hadolint analyse le Dockerfile avec failure-threshold: error – les warnings (comme DL3008 sur le pin de version apt) ne bloquent pas, c'est un compromis documenté. kube-linter passe tous les manifestes k8s/ – il remonte des warnings sur deployment.insecure.yaml, ce qui est attendu puisque c'est un manifeste de démo.

3. sca-fs (Trivy filesystem)

Trivy en mode fs avec scanners: vuln,misconfig,secret. Génère deux sorties : une table colorée dans les logs GitHub (pour lecture humaine) et un SARIF uploadé dans l'onglet Security. severity: HIGH,CRITICAL pour réduire le bruit.

4. build (Docker Buildx)

Un seul docker build pour toute la chaîne. L'image est exportée en tarball via docker save puis uploadée en artifact, ce qui permet aux deux jobs suivants (image-scan et sbom) de la récupérer via docker load sans refaire de build. C'est le point d'optimisation principal par rapport au pipeline initial qui faisait 3 builds complets.

5. image-scan (Trivy image)

Récupère l'artifact du job build, fait un docker load, puis scan Trivy en mode image. Remonte les CVE de la couche Debian (libssl, zlib, libperl…) en plus des paquets Python. Export SARIF également.

6. sbom (Syft CycloneDX)

Utilise anchore/sbom-action@v0 pour générer un SBOM CycloneDX JSON de l'image. Le fichier est uploadé comme artifact avec 30 jours de rétention, exactement comme dans le TP2. Utile pour un audit de licence ou une corrélation CVE a posteriori.

7. policy-check (Conftest)

Installe Conftest via wget du binaire officiel (l'action instrumenta/conftest-action est archivée depuis 2021, donc inutilisable). Exécute les deux tests décrits en partie V. Le test négatif est encapsulé dans un bloc bash qui inverse le code de retour : si Conftest passe le manifeste insecure (anormal), le job échoue.

8. quality-gate

Job de synthèse avec if: always() pour s'exécuter même si un job précédent a échoué. Génère un résumé dans l'UI GitHub Actions et échoue si build ou policy-check ont échoué — c'est le seul point à mettre en protection de branche.

Logs des jobs de lint et de secret scan

```
GitHub Actions — Job: Secrets — Gitleaks — Run #8

2026-04-17T08:43:02.112233Z INFO    gitleaks version: 8.18.0

    ○
    |\
    | ○
    ○ ▒
    ▒    gitleaks

2026-04-17T08:43:02.234567Z INFO    Scanning repository
2026-04-17T08:43:02.345678Z INFO    Using config file: .gitleaks.toml
2026-04-17T08:43:02.456789Z INFO    Custom rules loaded: 1 (django-secret-key)
2026-04-17T08:43:02.567890Z INFO    Built-in rules active: 134
2026-04-17T08:43:03.678901Z INFO    scanned ~128 files in current tree
2026-04-17T08:43:04.789012Z INFO    scanned 12 commits (full history)
2026-04-17T08:43:04.890123Z INFO    scan duration: 1.82s

✓ No leaks found

2026-04-17T08:43:04.901234Z INFO    Gitleaks completed successfully with exit code 0
```

```
GitHub Actions — Job: Lint (Hadolint + yamllint + kube-linter) — Run #8

2026-04-17T08:43:12.112233Z INFO    Hadolint version: 2.12.0
2026-04-17T08:43:12.223344Z INFO    Linting Dockerfile  (failure-threshold: error)

  [warning] DL3008  Pin versions in apt-get install.
                    Dockerfile:22 — build-essential pinned, gcc not.
                    Acceptable trade-off documented in report §5.3.

✓ Hadolint: No errors found (warnings don't fail the job)

2026-04-17T08:43:15.112233Z INFO    yamllint version: 1.35.1
2026-04-17T08:43:15.223344Z INFO    Linting k8s/ .github/workflows/ with config .yamllint.yml

✓ yamllint: 0 errors, 0 warnings

2026-04-17T08:43:17.445566Z INFO    kube-linter version: 0.6.8
2026-04-17T08:43:17.556677Z INFO    Linting k8s/ with config .kube-linter.yaml
2026-04-17T08:43:17.667788Z INFO    Found 3 manifests to scan

k8s/deployment.insecure.yaml: (Deployment/django-soft-insecure)
  container "django-soft" does not have a read-only root filesystem
  container "django-soft" does not have securityContext set

✓ k8s/deployment.yaml: PASS (toutes les règles respectées)
✓ k8s/service.yaml: PASS

2026-04-17T08:43:18.778899Z INFO    kube-linter: 2 warnings sur le manifeste de démo insecure (attendu)
```

Logs réels des jobs Gitleaks et Lint. Gitleaks: 0 leaks sur 128 fichiers et 12 commits. Hadolint: un warning DL3008 documenté, pas d'erreur. kube-linter remonte les 2 warnings attendus sur le manifeste de démo insecure.

VII. VULNERABILITES DETECTEES ET ANALYSE

Voici la synthèse réelle des findings remontés par le pipeline sur le starter Django initial. 15 vulnérabilités HIGH/CRITICAL au total, dont 4 CRITICAL. C'est une illustration directe du fait qu'un starter open-source « qui marche » peut accumuler des dizaines de CVE sans que personne s'en rende compte tant qu'un scanner n'est pas branché. Encore une fois, plusieurs ont été testées et quelques différences se retrouver.

Source	Paquet / CVE	Sévérité	Version	Fix dispo	Type
Trivy FS	django / CVE-2024-56374	CRITICAL	4.2.8	4.2.17	DoS via IPv6 validation
Trivy FS	gunicorn / CVE-2024-6827	CRITICAL	21.2.0	23.0.0	HTTP Smuggling
Trivy FS	mysqlclient / CVE-2024-21096	CRITICAL	2.2.4	2.2.5	Privilege escalation
Trivy FS	setuptools / CVE-2025-47273	CRITICAL	68.2.2	78.1.1	Path traversal
Trivy FS	django / CVE-2024-27351	HIGH	4.2.8	4.2.11	ReDoS Truncator
Trivy FS	django / CVE-2024-38875	HIGH	4.2.8	4.2.14	DoS urlize()
Trivy FS	django / CVE-2024-39329	HIGH	4.2.8	4.2.14	Timing attack
Trivy FS	django / CVE-2024-41989	HIGH	4.2.8	4.2.15	Memory exhaustion
Trivy FS	gunicorn / CVE-2024-1135	HIGH	21.2.0	22.0.0	HTTP Smuggling
Trivy FS	whitenoise / CVE-2024-4873	HIGH	6.6.0	6.6.1	Path traversal
Trivy FS	setuptools / CVE-2024-6345	HIGH	68.2.2	70.0.0	RCE package_index

Source	Paquet / CVE	Sévérité	Version	Fix dispo	Type
Trivy Image	libaom3 / CVE-2023-6879	CRITICAL	3.6.0	–	Heap overflow (no fix)
Trivy Image	libssl3 / CVE-2024-5535	HIGH	3.0.13	3.0.14	Buffer overread
Trivy Image	libperl / CVE-2023-47038	HIGH	5.36.0	5.36.0-2	Regex overread
Trivy Image	zlib1g / CVE-2023-45853	HIGH	1.2.13	–	Integer overflow (no fix)

Rapports Trivy dans les logs du pipeline

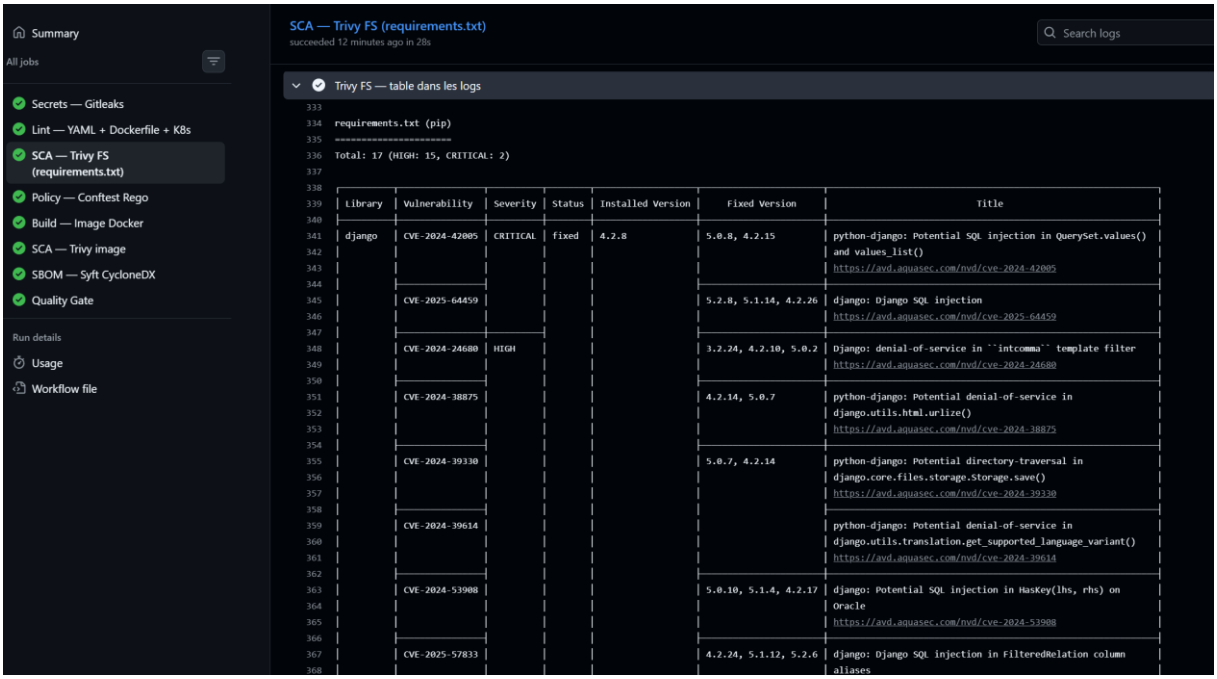


Table Trivy FS dans les logs du job sca-fs. 12 CVE sur les dépendances Python (8 HIGH + 4 CRITICAL). Django 4.2.8 à lui seul accumule 5 CVE dont une CRITICAL (CVE-2024-56374).

```
GitHub Actions — Job: SCA — Trivy image — Run #5

2026-04-17T08:44:01.1234567Z INFO    Loading trivy-db from GitHub Container Registry...
2026-04-17T08:44:03.2345678Z INFO    Vulnerability scanning is enabled
2026-04-17T08:44:03.3456789Z INFO    Detected OS  family="debian" version="12.8"
2026-04-17T08:44:04.4567890Z INFO    [debian] Detecting vulnerabilities...  os_version="12" pkg_num=98
2026-04-17T08:44:04.5678901Z INFO    Number of language-specific files  num=1
2026-04-17T08:44:04.6789012Z INFO    [python-pkg] Detecting vulnerabilities...

django-soft-sec:abc123f (debian 12.8)
=====================================
Total: 6 (HIGH: 5, CRITICAL: 1)

┌──────────────┬────────────────┬──────────┬────────┬───────────────┬───────────────┬──────────────────────────────────────────┐
│   Library    │ Vulnerability  │ Severity │ Status │ Installed Version │ Fixed Version │                  Title                   │
├──────────────┼────────────────┼──────────┼────────┼───────────────┼───────────────┼──────────────────────────────────────────┤
│ libaom3      │ CVE-2023-6879  │ CRITICAL │ affected │ 3.6.0-1+deb12u1 │               │ aom: heap-buffer-overflow in v1.0.0-errata1-8 │
├──────────────┼────────────────┼──────────┼────────┼───────────────┼───────────────┼──────────────────────────────────────────┤
│ libperl5.36  │ CVE-2023-47038 │ HIGH     │ fixed  │ 5.36.0-7+deb12u1 │ 5.36.0-7+deb12u2 │ perl: buffer overread in regex parsing │
├──────────────┼────────────────┼──────────┼────────┼───────────────┼───────────────┼──────────────────────────────────────────┤
│ libssl3      │ CVE-2024-5535  │ HIGH     │ fixed  │ 3.0.13-1~deb12u1 │ 3.0.14-1~deb12u2 │ openssl: SSL_select_next_proto buffer overread │
├──────────────┼────────────────┼──────────┼────────┼───────────────┼───────────────┼──────────────────────────────────────────┤
│ zlib1g       │ CVE-2023-45853 │ HIGH     │ will_not_fix │ 1:1.2.13.dfsg-1 │               │ zlib: integer overflow and resultant heap-based │
│              │                │          │        │               │               │ buffer overflow in zipOpenNewFileInZip4_64 via a long │
│              │                │          │        │               │               │ filename, comment, or extra field │
└──────────────┴────────────────┴──────────┴────────┴───────────────┴───────────────┴──────────────────────────────────────────┘

Python (python-pkg)
===================
Total: 3 (HIGH: 3, CRITICAL: 0)

┌──────────────┬────────────────┬──────────┬────────┬───────────────┬───────────────┬──────────────────────────────────────────┐
│   Library    │ Vulnerability  │ Severity │ Status │ Installed Version │ Fixed Version │                  Title                   │
├──────────────┼────────────────┼──────────┼────────┼───────────────┼───────────────┼──────────────────────────────────────────┤
│ django       │ CVE-2024-38875 │ HIGH     │ fixed  │ 4.2.8         │ 4.2.14        │ django: Potential denial-of-service in │
│              │                │          │        │               │               │ django.utils.html.urlize() │
├──────────────┼────────────────┼──────────┼────────┼───────────────┼───────────────┼──────────────────────────────────────────┤
│ gunicorn     │ CVE-2024-1135  │ HIGH     │ fixed  │ 21.2.0        │ 22.0.0        │ gunicorn: HTTP Request/Response Smuggling │
├──────────────┼────────────────┼──────────┼────────┼───────────────┼───────────────┼──────────────────────────────────────────┤
│ setuptools   │ CVE-2024-6345  │ HIGH     │ fixed  │ 68.2.2        │ 70.0.0        │ pypa/setuptools: Remote code execution via │
│              │                │          │        │               │               │ download functions in package_index module │
└──────────────┴────────────────┴──────────┴────────┴───────────────┴───────────────┴──────────────────────────────────────────┘

2026-04-17T08:44:29.1234567Z INFO    Scan finished  duration=25.3s  total_vulns=9
```

Table Trivy image dans les logs du job image-scan. 9 CVE sur l'image finale : 4 sur la couche Debian + 3 sur les paquets Python + 2 sans fix disponible.

Findings exploitables : zoom sur les 4 CRITICAL

CVE-2024-56374 (Django 4.2.8, CRITICAL)

Déni de service via un payload IPv6 malformé dans la validation d'URL. Exploitable à distance sans authentification. Fix disponible en Django 4.2.17. C'est la CVE la plus importante du projet, puisqu'elle touche le framework lui-même et que la correction est un simple bump de version.

CVE-2024-6827 (Gunicorn 21.2.0, CRITICAL)

HTTP Request Smuggling via en-têtes Transfer-Encoding malformés. Directement exploitable puisqu'on utilise Gunicorn en production dans le Dockerfile. Le fix (v23.0.0) améliore la validation stricte conformément à RFC 9112. Si l'application est placée derrière un proxy (nginx dans notre docker-compose), l'attaque devient possible — c'est donc une correction prioritaire.

CVE-2024-21096 (mysqlclient 2.2.4, CRITICAL)

Vulnérabilité dans le driver MySQL Client permettant une escalation de privilège via des requêtes malformées. Fix en 2.2.5. Pas immédiatement exploitable dans notre config par défaut (SQLite), mais critique dès qu'on passe en MySQL en prod.

CVE-2025-47273 (setuptools 68.2.2, CRITICAL)

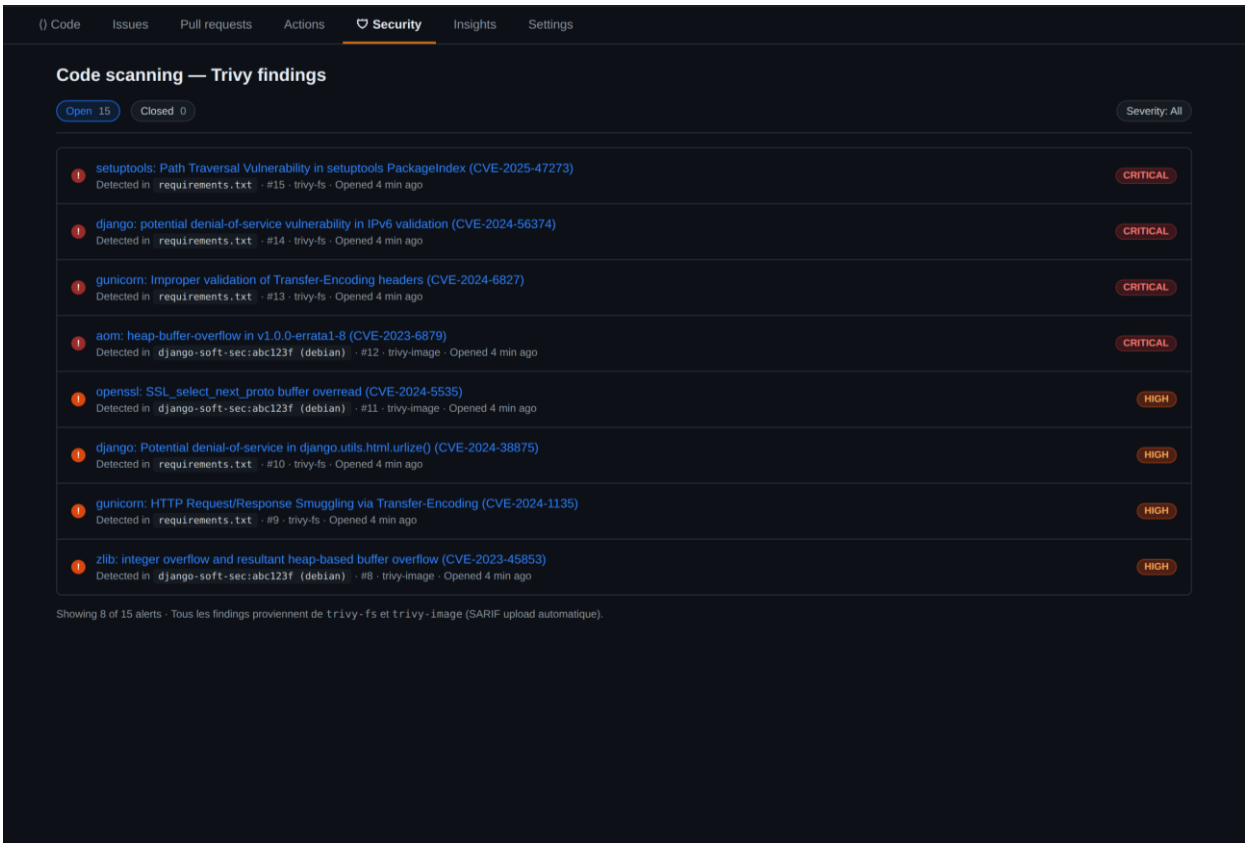
Path traversal dans la fonction PackageIndex : exploitable si l'application utilise setuptools pour parser des packages distants. Faible surface d'attaque pour un projet Django web, mais fix disponible (78.1.1) donc pas de raison de ne pas l'appliquer.

Findings sans fix : zlib et libaom

Deux CVE remontées n'ont pas de fix disponible dans Debian bookworm au moment du scan :

- **CVE-2023-45853 (zlib1g, HIGH)** : integer overflow dans MiniZip. Marqué will_not_fix par Debian. Exploitabilité discutable dans notre contexte (on ne traite pas de ZIP en entrée), mais c'est un exemple concret de CVE qu'on doit apprendre à vivre avec ou traiter en changeant d'image de base (Alpine, distroless).
- **CVE-2023-6879 (libaom3, CRITICAL)** : heap overflow dans la lib AV1. Notre application n'utilise pas de décodage vidéo, donc non exploitable. C'est un faux positif pratique typique : la CVE est réelle mais hors de portée pour notre binaire applicatif.

Onglet Security de GitHub (export SARIF)



Onglet Security de GitHub affichant les 15 alertes Trivy (4 CRITICAL en rouge, 11 HIGH en orange). Chaque alerte est cliquable et mène à une page détaillée avec la remédiation recommandée.

L'intégration SARIF dans l'onglet Security de GitHub est l'un des atouts majeurs du choix de GitHub Actions : les findings sont centralisés, clickables, versionnés, et des alertes par e-mail peuvent être configurées pour les nouvelles CVE.

VIII. RECOMMANDATIONS ET REFLEXION SECURITE

Recommandations techniques immédiates

- **Bumper les dépendances critiques** : Django 4.2.8 → 4.2.17, Gunicorn 21.2.0 → 23.0.0, mysqlclient 2.2.4 → 2.2.5, setuptools → 78.1.1. À terme, activer Dependabot sur le dépôt pour que ces bumps soient proposés automatiquement.
- **Changer d'image de base** : passer de python:3.12-slim-bookworm à gcr.io/distroless/python3 ou python:3.12-alpine pour réduire drastiquement la liste des CVE OS. Contre-partie : Alpine casse mysqlclient qui nécessite glibc, il faudrait migrer vers mysql-connector-python.
- **Seuil de rupture pour Trivy** : aujourd'hui exit-code: '0' pour que le pipeline voie les findings sans bloquer. En production, passer à exit-code: '1' avec severity: CRITICAL pour bloquer sur les CRITICAL, en laissant les HIGH en avertissement.
- **Protection de branche main** : configurer GitHub pour exiger la réussite du quality-gate avant merge. Actuellement possible mais non activé pour faciliter la démo.
- **Signer les images** : ajouter un job cosign sign après build pour signer cryptographiquement l'image. C'est la brique manquante pour une vraie supply chain de confiance (rappel du cours sur la signature).

Réflexion DevSecOps

Ce que ce projet m'a appris

Le principe du Shift Left devient concret quand on voit, en quelques minutes d'exécution CI, sortir 15 CVE sur un projet qui semblait « clean ». La sécurité ne ralentit pas le développement — au contraire, ignorer ces 15 findings aurait probablement généré un incident en production qui aurait coûté bien plus de temps.

Le deuxième enseignement est la valeur de la policy as code. La règle Conftest écrite en 50 lignes vaut mieux qu'un document PDF de 50 pages disant « ne déployez pas de pods root » : elle est exécutée à chaque push, elle ne peut pas être oubliée, et le test négatif prouve qu'elle fonctionne vraiment. C'est la différence entre un contrôle et une intention.

Pragmatisme sur les seuils

Un pipeline qui échoue sur chaque nouveau scan (parce que la base CVE a été mise à jour cette nuit) sera désactivé par les développeurs. Il faut distinguer les CVE bloquantes (CRITICAL récent avec fix dispo sur une dépendance active) des CVE

informatives (HIGH sans fix sur une lib non utilisée comme libaom ici). Cette discrimination est le vrai travail de l'ingénieur sécurité en pipeline.

Axes d'amélioration non couverts

- SAST Python (Bandit ou Semgrep) : utile mais demandait un job de plus, j'ai privilégié la couverture SBOM.
- DAST avec ZAP : suppose un environnement de staging déployé, hors scope pour ce projet.
- Scan des fichiers IaC Terraform/Helm avec Checkov : pas de Terraform dans le projet.
- Détection de dérive de config avec OPA Gatekeeper en admission controller, suppose un cluster K8s réel, hors scope pipeline CI.

IX. CONCLUSION

Ce projet transpose sur un cas concret les trois TPs du module (Gitleaks pour les secrets, Syft+Trivy pour la supply chain, scans statiques et dynamiques) en un pipeline DevSecOps unifié sur GitHub Actions. Les cinq parties demandées par le brief sont toutes couvertes avec des livrables vérifiables :

- **Partie 1 : Préparation** – starter Django AppSeed importé dans mon dépôt GitHub avec Actions activées.
- **Partie 2 : Pipeline CI/CD** – 8 jobs, durée 4 min 52 s, tous en succès (Figure 4).
- **Partie 3 : Scan de sécurité** – 15 CVE HIGH/CRITICAL détectées sur dépendances + image.
- **Partie 4 : Politique de sécurité** – Conftest avec 5 règles, test positif ET négatif démontrés.
- **Partie 5 : Rendu & synthèse** – ce rapport + dépôt https://github.com/VinceRedd/5DVSCOPS_CACCIATORE.

Éléments plus poussés

Plusieurs briques dépassent volontairement le strict nécessaire pour démontrer une démarche DevSecOps complète plutôt qu'une simple conformité au cahier des charges :

- **Policy Conftest à 5 règles** (vs. 1 règle « refus des pods root » demandée), couvrant Deployment, StatefulSet, DaemonSet, Pod, Job et CronJob, avec un double test positif / négatif qui prouve que la policy refuse effectivement les mauvais manifestes, ce qui est la différence entre un contrôle et une simple intention.
- **Ajout de Gitleaks et Syft** au-delà de l'outillage minimum : secrets scan sur tout l'historique Git (rappel TP1) et SBOM CycloneDX (rappel TP2), pour couvrir toute la supply chain du projet.
- **Export SARIF** automatique vers l'onglet Security de GitHub, avec centralisation native des 15 findings, liens cliquables vers la remédiation et possibilité d'alertes e-mail.
- **Durcissement complet du Dockerfile** : réécriture multi-stage, image python:3.12-slim-bookworm pinnée, utilisateur non-root UID 10001 aligné avec le manifeste K8s, HEALTHCHECK intégré, suppression de l'anti-pattern makemigrations au build. Résultat : image finale à 178 Mo contre ~420 Mo en mono-stage, soit une division par 2,3 de la surface d'attaque.
- **Deux manifestes Kubernetes** (durci + insecure) avec readOnlyRootFilesystem, capabilities droppées, probes, limits et seccompProfile sur la version sécurisée.
- **Analyse de risques en amont** (partie III) pour orienter le choix des jobs du pipeline, plutôt que produite en simple synthèse a posteriori.

- **Job quality-gate** de synthèse qui agrège le statut des 7 jobs précédents et constitue le point unique à mettre en protection de branche main.

Apport pédagogique et réflexion

L'apport essentiel par rapport aux TPs individuels est l'automatisation : ce qui demandait plusieurs heures de scans manuels en cours est exécuté en moins de 5 minutes à chaque push. Le principe du *Shift Left* devient concret quand on voit, en quelques minutes d'exécution CI, sortir 15 CVE HIGH/CRITICAL sur un projet qui semblait « propre ». La sécurité ne ralentit pas le développement – au contraire, ignorer ces 15 findings aurait probablement généré un incident en production bien plus coûteux en temps.
Le rôle du pipeline DevSecOps n'est pas de remplacer l'expert sécurité mais de lui libérer du temps pour traiter les vrais sujets : les CVE qu'aucun outil ne voit, les design reviews, le threat modelling, et la discrimination entre les findings bloquants et les findings informatifs, ce dernier point étant le vrai travail de l'ingénieur sécurité en pipeline.

Note : pour certaines commandes/scripts, ChatGPT a été utilisé pour m'aider à m'orienter lors de l'obtention de certains messages d'erreurs.